음성 AI 전문가 과정 · Day 17 / 40

Day 17

경량 TTS: Kokoro
8,200 만 파라미터로 CPU 실시간 합성

설치 · 추론 → misaki 음소 → pred_dur · 자막 → voice pack · 섞기 → RTF · TTFA · 스트리밍 → 한국어 실험
4 주차 M4 · 4 시간 · CPU 로 충분 (GPU 는 비교용) · 실습 노트북 Part 0 ~ 7

```
text   Kokoro is a lightweight model.
misaki kəkˈɔɹO ɪz ɐ lˈItwˌAt mˈɑdᵊl.
voice  af_heart → pack[len(ps) − 1]
KModel pred_dur × 600 → 24 kHz 파형

CPU    RTF …    첫 소리 (TTFA) … 초

한국어 K1 · K2 · V  평균 CER …
```

출력 모양 예시 · 숫자는 각자 실행 결과

0.0 · 오늘 답할 질문

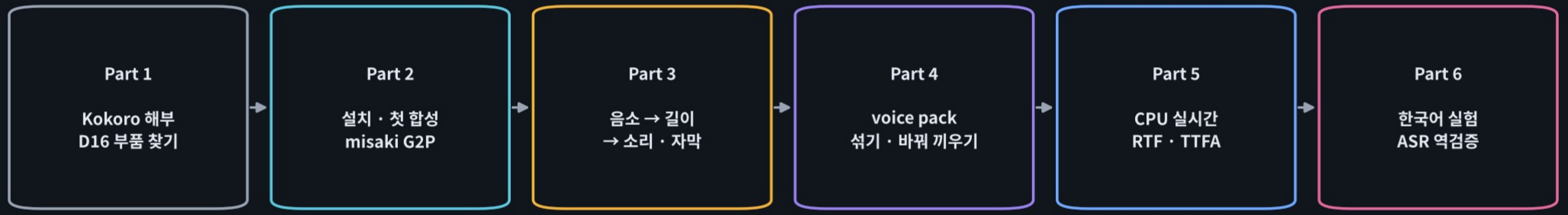
작은 모델이 어떻게 좋은 목소리를 낼까

1	Kokoro 안에서 D16 의 부품 (G2P · duration · Vocoder) 은 어디에 있나 ?	Part 1
2	파이프라인과 모델은 왜 나뉘어 있고 , misaki 는 무슨 음소를 만드나 ?	Part 2
3	기호마다 몇 프레임 ? 단어 자막 시간은 어떻게 얻나 ?	Part 3
4	목소리는 어디에 들어 있고 , 어떻게 섞나 ?	Part 4
5	CPU 에서 " 실시간 " 은 무엇으로 재고 , 첫 소리를 어떻게 앞당기나 ?	Part 5
6	한국어 목소리가 없는 모델에 한국어 음소를 넣으면 ?	Part 6

▶ 애니메이션 D17-1 오늘의 흐름

설치 → 합성 → 들여다보기 → 목소리 → 속도 → 한국어

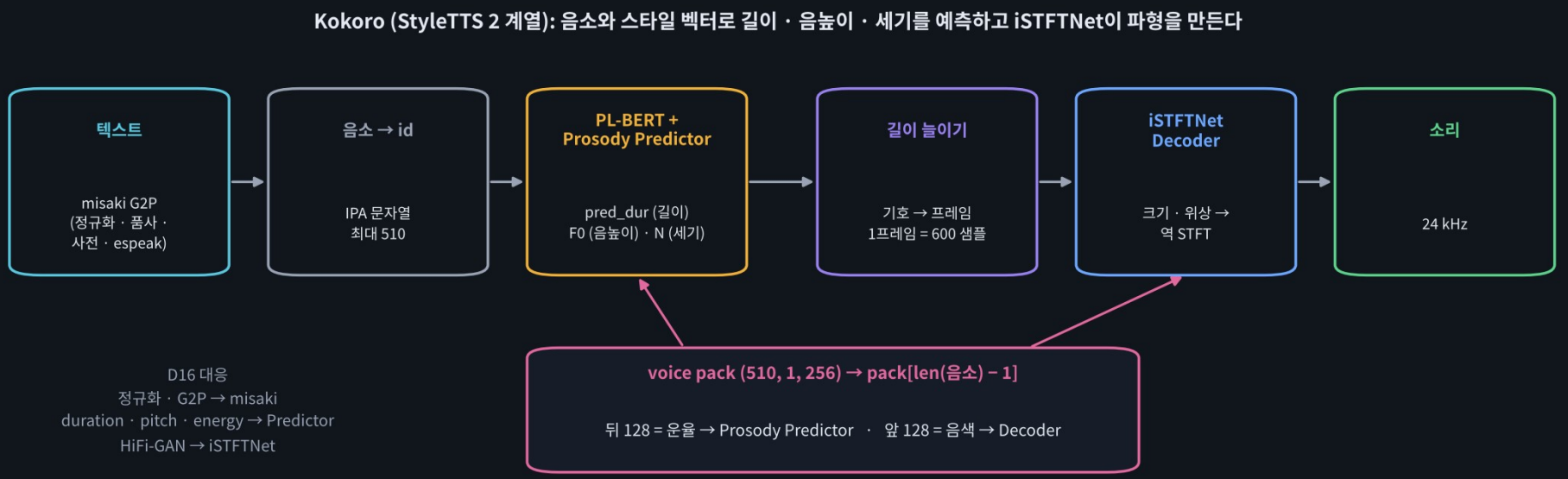
D17 · 작은 모델 하나로 여러 목소리를, CPU에서 실시간으로



Kokoro-82M · StyleTTS 2 + iSTFTNet · Apache 2.0 · 24 kHz · 8개 언어 54개 목소리 (한국어 없음)

교시	내용	실습
1 교시	Kokoro 구조, KPipeline · KModel, 첫 합성 · RTF, misaki 음소	Part 0 ~ 2 · 빈칸 1 · 2
2 교시	음소 → id → pred_dur → 파형, 단어 타임스탬프 · SRT, speed	Part 3 · 빈칸 3 · 4
3 교시	voice pack (510, 1, 256), 목소리 섞기, 음색 · 운율 바꿔 끼우기	Part 4 · 빈칸 5 · 6
4 교시	RTF · TTFA · 스레드 · 스트리밍, 한국어 실험 (K1 · K2 · V), 정리	Part 5 ~ 7 · 빈칸 7 ~ 9 · 노션

StyleTTS 2 + iSTFTNet: 음소 → 운율 예측 → 파형



D16 에서 배운 것	Kokoro 안의 부품
정규화 + G2P	misaki (spaCy 품사 + 사전 + espeak-ng)
Duration · Pitch · Energy	Prosody Predictor → pred_dur · F0 · N
Vocoder (HiFi-GAN)	iSTFTNet: 크기 · 위상 → 역 STFT
화자 · 스타일	voice pack 256 = 음색 128 + 운율 128

▶ 애니메이션 D17-2 Kokoro 해부

작고 · 빠르고 · 상업 사용 가능

	D16 MMS-TTS (VITS)	Kokoro-82M
파라미터	약 3,600 만	약 8,200 만
입력	로마자 (uroman)	음소 (IPA)
언어 · 목소리	1,100 여 언어 · 언어마다 1 개	8 개 언어 · 54 개 목소리 (한국어 없음)
샘플링	16 kHz	24 kHz
라이선스	CC BY-NC 4.0	Apache 2.0

목소리는 모델 안이 아니라 voice pack 파일에 → 모델 하나로 54 개 목소리

🍎 밀키트 요리사

요리사 (KModel) 는 재료 (음소) 만 받으면 요리하고 , 양념 세트 (voice pack) 를 바꾸면 맛 (목소리) 이 바뀝니다 . 재료 손질 (G2P) 은 언어마다 다른 보조 요리사 (misaki).

모델은 하나 , 파이프라인은 언어마다

```
kmodel = KModel(repo_id=REPO).to("cpu").eval()
pipe = KPipeline(lang_code=____, repo_id=REPO,   # 빈칸 1
                 model=kmodel)
for r in pipe(TEXT, voice="af_heart"):
    audio = r.audio                       # 24 kHz
RTF = gen_time / (len(audio) / ____)   # 빈칸 2
```

🧩 빈칸 실습 1 · 2

빈칸 1 → "a" (미국 영어)

빈칸 2 → SR_K (24000)

lang_code	언어	목소리 예 (등급)	G2P
a	미국 영어	af_heart (A) · af_bella (A−) · am_michael (C+)	misaki
b	영국 영어	bf_emma (B−) · bm_george (C)	misaki
j · z	일본어 · 중국어	jf_alpha (C+) · zf_xiaoxiao (D)	misaki[ja · zh]
e f h i p	스페인 · 프랑스 · 힌디 · 이탈리아 · 포르투갈어	ff_siwis (B−) 등	espeak-ng

숫자 · 품사 · 사전 · espeak-ng 순서로 음소를 만든다

입력	misaki 결과
$12.50	twelve dollars and fifty cents
I read it yesterday.	ɹˈɛd (과거)
I will read it.	ɹˈid (현재)
St. James St.	sənt ... sənt × (둘째는 street)
3:15 PM	θɹˈiːfˈɪftin pˌiˈɛm

기호	뜻
' · ˌ	1 차 · 2 차 강세
A · I	eɪ (say) · aɪ (my)
O · W	oʊ (go) · aʊ (now)
Y · T	ɔɪ (boy) · 탄설음 ɾ

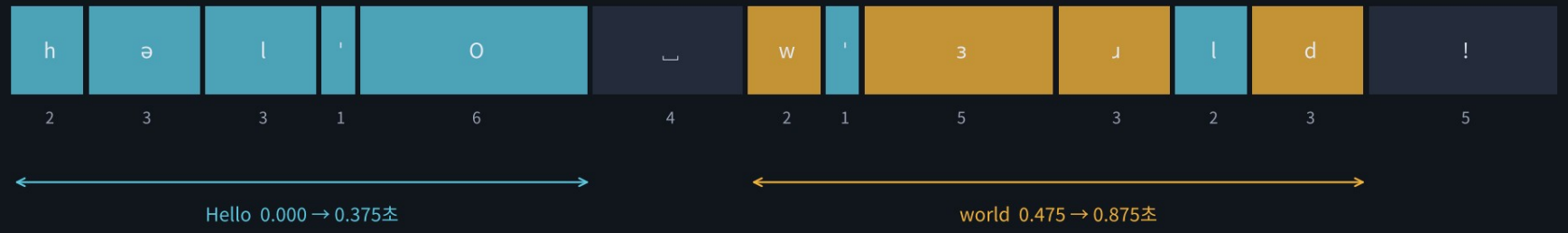
G2P 오류는 재학습 없이 정규화 규칙 · 음소 직접 지정으로 고친다

▶ 애니메이션 D17-3 misaki 음소 읽기

소리 길이 (샘플) = pred_dur 합 × 600

pred_dur: 기호마다 프레임 수 → 1프레임 = 600 샘플(25 ms) → 단어 시작 · 끝 시간(자막)

pred_dur 합 40 프레임 × 600 = 24,000 샘플 = 1.000초 · speed = 2 → pred_dur ÷ 2



업샘플 10 × 6 × iSTFT hop 5 × 내부 2배 = 600 (D16: 업샘플 곱 = hop)

1. 음소 → id (모르는 기호는 조용히 버림), 앞뒤 경계 0 → 최대 512
2. PL-BERT 문맥 → Prosody Predictor: pred_dur = round(duration ÷ speed)
3. 기호 특징을 pred_dur 만큼 복사 (D16 length regulator) → F0 · N 예측
4. iSTFTNet: 크기 · 위상 → 역 STFT → 파형

▶ 애니메이션 D17-4 길이와 타임스탬프

프레임 수를 더하면 자막 시간이 나온다

```
r = next(pipe(TEXT, voice="af_heart"))
FRAME = ____                          # 빈칸 3
assert len(r.audio) == r.pred_dur.sum() * FRAME
for t in r.tokens:
    print(srt_time(t.____),           # 빈칸 4
          "-->", srt_time(t.end_ts), t.text)
```

🧩 빈칸 실습 3·4

빈칸 3 → 600

빈칸 4 → start_ts

```
00:00:00,000 --> 00:00:00,…  Kokoro
00:00:00,… --> 00:00:00,…  is
…
```

속성	내용
r.phonemes	음소 문자열
r.pred_dur	기호별 프레임 수 (경계 포함)
t.start_ts · t.end_ts	단어 시작 · 끝 (초)
r.audio	24 kHz 파형

빠르게 해도 목소리 높이는 그대로

speed	pred_dur 합	길이	음높이
0.8	× 1.25	길어짐	거의 같음
1.0	기준	기준	기준
1.5	× 0.67	짧아짐	거의 같음
2.0	× 0.5	절반	거의 같음

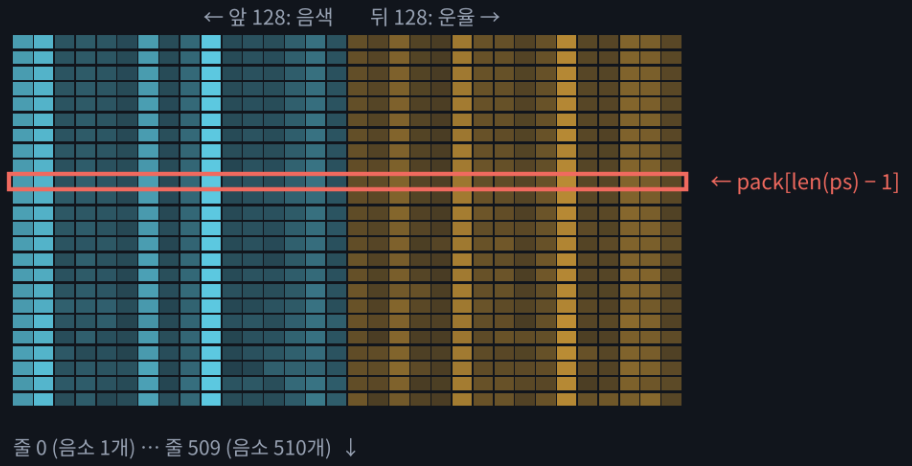
HiFi-GAN
업샘플 합성곱으로 파형 샘플을 끝까지 직접 만든다

iSTFTNet (Kokoro)
업샘플은 일부만 , 마지막은 작은 STFT 의 크기 · 위상을 예측해 역 STFT → 계산이 가볍다

speed 는 길이 예측만 바꾼다 : pred_dur = round(duration ÷ speed)

목소리 하나 = (510, 1, 256) 숫자 묶음

voice pack = 음소 수별 스타일 510줄 × 256 (음색 128 + 운율 128)



앞 128
→ Decoder (음색 · 결)

뒤 128
→ Predictor (길이 · F0)

섞기: (1 − w) · A + w · B
쉼표: pipe(text, voice="af_heart,am_michael") = 평균
바꿔 끼우기: cat(A[:, :128], B[:, 128:])

🍎 **양념 세트**

요리 양 (음소 수) 별로 510 가지 배합 , 각 배합은 향 (음색) 반 + 간 (운율) 반 . 두 상자를 섞으면 중간 맛 .

▶ 애니메이션 D17-5 voice pack 탐색기

ref_s = pack[len(음소) − 1]

```
pack = pipe.load_voice("af_heart")   # (510, 1, 256)
ps = r.phonemes
ref_s = pack[____ - 1]               #  빈칸 5
out = kmodel(ps, ref_s, speed=1)
timbre, prosody = ref_s[:, :128], ref_s[:, 128:]
```

🧩 빈칸 실습 5

빈칸 5 → len(ps)

부분	들어가는 곳
앞 128	Decoder → 음색
뒤 128	Predictor → 길이 · F0

목소리	등급	특징 (노트북 표에서 F0 중앙값 확인)
af_heart	A	미국 여성 · 대표 목소리
af_bella	A−	미국 여성
am_michael	C+	미국 남성
bf_emma	B−	영국 여성

숫자이므로 평균을 내면 목소리가 섞인다

```
pa, pb = pipe.load_voice("af_heart"), pipe.load_voice("am_michael")
def blend(w):
    return (1 - w) * pa + w * ____     # 빈칸 6

torch.allclose(blend(0.5), pipe.load_voice("af_heart,am_michael"))
```

▶ 애니메이션 D17-8 목소리 섞기

🧩 빈칸 실습 6

빈칸 6 → pb

w	결과
0.0	af_heart 그대로
0.3	여성 쪽 중간
0.5	쉼표 평균과 같음
1.0	am_michael

A 음색 + B 운율 = ?

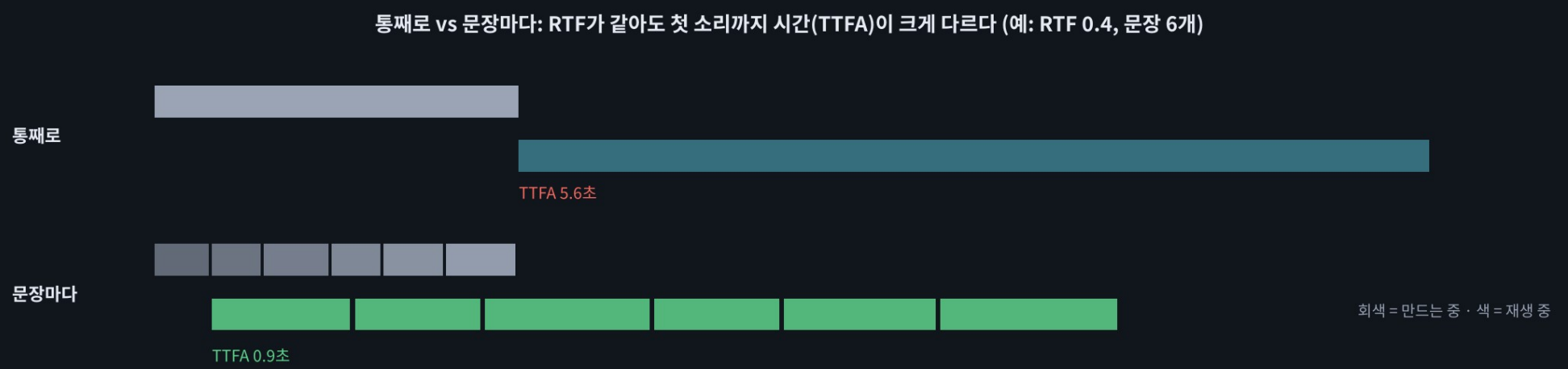
조합	프레임 (길이)	음높이 (F0)	음색
A 그대로	A	A (높음)	A
B 그대로	B	B (낮음)	B
A 음색 + B 운율	B	B 쪽 (낮아짐)	A
B 음색 + A 운율	A	A 쪽	B

```
swap = torch.cat([
  ra[:, :128],   # A 음색
  rb[:, 128:]],  # B 운율
  dim=1)
```

가까운 목소리끼리 섞을수록 자연스럽다 · 섞어도 라이선스와 윤리는 그대로 확인

▶ 애니메이션 D17-6 스트리밍 타임라인

RTF < 1 그리고 짧은 TTFA



💬 **쉽게 말하면** 첫 문장이 끝나자마자 재생을 시작하고 , 재생하는 동안 다음 문장을 만든다

끊김 없음 조건 : 조각 i 준비 시각 ≤ TTFA + 앞 조각 길이의 합

측정하고, 문장마다 흘려보낸다

```
LINES = re.sub(r"(?<=\.) ", "\n", PARA)   #  문장마다 줄바꿈
def stream_log(split):
    for r in pipe(LINES, voice="af_heart", split_pattern=split):
        log.append( 준비_시각 ,  길이 ,  글자 )
whole  = stream_log(None)
stream = stream_log(____)                  #  빈칸 7
```

🧩 빈칸 실습 7

빈칸 7 → r"\n+"

장치	측정
CPU 1 스레드	RTF · 생성 초
CPU 전체	RTF · 생성 초
GPU (선택)	RTF · 생성 초

방식	조각	첫 소리까지	끊김
통째로 (None)	1	문단 전체 생성 시간	—
문장마다 (r"\n+")	문장 수	첫 문장 생성 시간	표의 끊김 열 확인

질문 끝 → 첫 소리 = ASR + LLM 첫 문장 + TTS 첫 조각

1. ASR 이 마지막 말을 받아쓴다 (D9 · D12)
2. LLM 이 답의 첫 문장을 내놓는다
3. TTS 가 첫 조각을 만든다 (첫 조각 길이 × RTF)
4. 재생하는 동안 다음 조각을 만든다

▶ 애니메이션 D17-9 실시간 예산 계산기

모아서 한 번에 합성 = TTFA 최악

줄이는 방법	효과
첫 조각을 짧게 (쉼표 · 문장 단위)	TTS 시간 ↓
스레드 = 코어 수	RTF ↓
LLM 답을 문장마다 TTS 로	기다림 ↓
버퍼를 조금 쌓고 시작	끊김 ↓

Kokoro 에 한국어 IPA 를 직접 넣어 보기



조건	처리 → 모델
K1	원문 → espeak-ng ko → Kokoro
K2	normalize_ko → g2pk2 → espeak → Kokoro
V	normalize_ko → g2pk2 → uroman → MMS VITS

```
VOCAB = set(kmodel.____)                    # 빈칸 9
dropped = {c for c in ps if c not in VOCAB}
pipe.____(ps, voice=KO_VOICE)               # 빈칸 8
```

🧩 빈칸 실습 8 · 9

빈칸 8 → generate_from_tokens

빈칸 9 → vocab

▶ 애니메이션 D17-7 한국어 음소 실험

정리 · 오늘 배운 것

작은 모델 + 좋은 음소 + 목소리 벡터 = 실시간 TTS

개념	한 줄	코드
구조	StyleTTS 2 + iSTFTNet, 82M, Apache 2.0, 24 kHz	`KModel`
파이프라인	언어별 G2P + voice 관리 , 모델 공유	`KPipeline(lang_code="a", model=kmodel)`
길이 · 자막	프레임 600 샘플 (25 ms), 단어 시작 · 끝	`r.pred_dur, t.start_ts`
voice pack	(510, 1, 256), 음색 128 + 운율 128	`pack[len(ps) - 1]`
섞기	선형 보간 · 쉼표 평균	`(1 - w) * pa + w * pb`
실시간	RTF < 1 + 짧은 TTFA, 문장 단위 스트리밍	`split_pattern=r"\n+"`
음소 직접 입력	G2P 를 건너뛰고 IPA 를 모델에	`pipe.generate_from_tokens(ps, voice)`

정리 · 다음 시간

D18 표현형 TTS

- ☐ Kokoro 부품을 D16 TTS 단계와 짝지을 수 있다
- ☐ KPipeline · KModel 로 합성하고 RTF 를 쟀다
- ☐ misaki 음소 · 강세 · 약속 기호를 읽었다
- ☐ pred_dur × 600 으로 길이 · 단어 자막을 만들었다
- ☐ voice pack 을 섞고 음색 · 운율을 바꿔 끼웠다
- ☐ TTFA 를 스트리밍으로 줄이고 한국어 실험을 했다

다음 : D18 표현형 TTS

감정 · 억양 제어 : Chatterbox 로 감정 세기를 조절하고 F5-TTS · CosyVoice 같은 오픈소스 모델을 비교합니다 . 오늘 본 " 음색과 운율을 나누는 생각 " 이 어떻게 발전했는지 , 한국어를 지원하는 표현형 모델을 같은 ASR 역검증으로 평가합니다 .

worksheet.html	40 문항
개념 워크시트 · 노션 제출	

animation.html	9 개 탭
인터랙티브 애니메이션	

D17_Kokoro_ 실습 .ipynb	빈칸 9
실습 노트북 · CPU	

온라인 자료 imguru-mooc.github.io/AI_Voice/day17